SpecDecode: Building a Speculative Decoding Inference Accelerator from Scratch

Prashant Gautam

Department of Industrial Engineering and Operations Research
Indian Institute of Technology Bombay

Roll No. 24B4526

github.com/PrashantGautam-alt/specdecode

June 2026

Abstract

Large language models generate text one token at a time, with each token requiring a full forward pass of the target model. This sequential dependency is the fundamental throughput bottleneck at inference time. This project implements two techniques that attack this bottleneck from different angles – speculative decoding and Medusa – building both from scratch on Llama 3.1/3.2 models and measuring the speedup on a 2x RTX A5000 GPU setup. Speculative decoding with K=4 achieves a 1.17x speedup over the 8B baseline. Medusa with a greedy 3-pass implementation runs at 0.69x due to per-round overhead; tree attention (implemented but not yet benchmarked due to server downtime) is expected to bring this above 1.5x by collapsing three backbone passes into one.

Motivation

The standard autoregressive loop is simple: run the model, take the last logit, sample a token, append it, repeat. At 38 tokens/sec on a Llama 3.1 8B model, generating a 500-token response takes about 13 seconds. This is slow enough to matter for user-facing applications.

The key observation in Leviathan et al. [1] is that the bottleneck is not compute but *sequential dependency*. The transformer can process multiple tokens in parallel during training (each position attends to all previous positions in one pass), but during generation we feed one token at a time because each token depends on the previous one. Speculative decoding breaks this dependency partially by letting a cheap draft model propose several tokens at once, then verifying all of them in a single target model pass.

Background

Autoregressive Generation

Given prompt tokens $x_1, \ldots, x_n$, the model generates token x_{n+1} by computing:

$$x_{n+1} \sim p_\theta(\cdot \mid x_1, \ldots, x_n)$$

Each step requires a full forward pass. With the KV cache, keys and values for positions $1 \ldots n$ are stored from the previous step, so only the new token needs to be processed. This makes each step $O(1)$ in compute but the sequential nature remains – step $t+1$ cannot begin until step t produces a token.

The KV Cache

Without caching, generating T tokens on a sequence of length n costs $O(T \cdot n)$ forward passes in terms of attention operations. With the KV cache, keys and values for all seen positions are stored and reused. Each new step only computes Q for the newest token, reduces attention operations to $O(T)$. The tradeoff is memory: for Llama 3.1 8B with sequence length 2048, the KV cache occupies roughly $2 \times 32 \times 2048 \times 128 \times 2 \approx 0.5$ GB per batch element in float16.

Speculative Decoding

Algorithm

Speculative decoding [1] uses two models: a small fast draft model q and a large accurate target model p. Each round:

1. **Draft:** Run the draft model autoregressively for K steps, collecting tokens $\tilde{x}_1, \ldots, \tilde{x}_K$ and their distributions $q(\cdot \mid \cdot)$.
2. **Verify:** Feed all K draft tokens to the target model in one forward pass. The transformer processes all positions in parallel, yielding K distributions $p(\cdot \mid \cdot)$ simultaneously.
3. **Accept:** For each draft token $\tilde{x}_i$ from left to right, accept with probability $\min(1, p(\tilde{x}_i)/q(\tilde{x}_i))$. Stop at the first rejection.

4. **Correct:** On rejection, sample the correction token from normalize($\max(0, p - q)$).

Rejection Sampling Correctness

The acceptance criterion $\min(1, p(x)/q(x))$ guarantees that the marginal distribution of each accepted token equals p exactly. This is not an approximation – speculative decoding is lossless by construction.

Proof sketch: let $\alpha = \min(1, p(x)/q(x))$. The probability of outputting token x is:

$$q(x) \cdot \alpha + (1 - A) \cdot r(x)$$

where $A = \sum_x q(x) \min(1, p(x)/q(x))$ is the overall acceptance probability and $r(x) = \max(0, p(x) - q(x))/(1 - A)$ is the residual distribution. Expanding and simplifying yields $p(x)$ for all x.

Empirically verified with a 100K trial test: maximum deviation 0.27% from target distribution p – within the expected $O(1/\sqrt{N})$ Monte Carlo error.

Cost Model

Each round costs K draft passes plus 1 target pass. If a draft pass costs fraction c of a target pass:

$$\text{speedup} \approx \frac{M}{c \cdot K + 1}$$

where M is the average tokens accepted per round. Empirically, $c \approx 0.40$ on the A5000 – each 1B draft pass costs about 40% of an 8B target pass. This is higher than the parameter ratio ($1/8 = 0.125$) because single-token forward passes are dominated by fixed per-call overhead, not by FLOPs.

At K=4 with $M = 3.05$: predicted speedup $= 3.05/(0.40 \times 4 + 1) = 1.17$x. This matched the measured value exactly.

K Sweep Results

Table 1: K sweep. Draft: Llama 3.2 1B Instruct. Target: Llama 3.1 8B Instruct. 100 tokens, 3 runs per K. Naive 8B baseline: 38.0 tok/s.

K	tok/s	Speedup	Avg tokens/round	Predicted speedup
1	27.8	0.73x	1.00	0.71x
2	38.4	1.01x	1.83	1.02x
4	**44.4**	**1.17x**	**3.05**	**1.17x**
6	39.0	1.03x	3.65	1.07x
8	34.0	0.89x	3.82	0.95x

K=4 is optimal because draft cost grows linearly and unconditionally (+0.40 per draft token regardless of acceptance), while acceptance gains are sublinear and conditional (one wrong token wastes all remaining draft passes that round). The cost model captures both effects and predicts the peak correctly.

A key finding: using the base 1B model as draft against the instruct 8B target peaked at only 0.97x. Switching to the instruct 1B draft raised acceptance from 2.45 to 3.05 at K=4. Base and instruct models operate in different "dialects" – base continues text, instruct responds like an assistant. Matching the training style of draft and target is as important as the model size ratio.

Medusa

Architecture

Medusa [2] eliminates the separate draft model entirely by attaching small MLP heads directly to the frozen backbone. Each head reads the final hidden state $h \in \mathbb{R}^d$ (where $d = 4096$ for Llama 8B) and predicts a token further in the future:

$$\text{head}_k(h) = W_2^{(k)}\big(\text{SiLU}(W_1^{(k)}h) + h\big)$$

Head k predicts the token at position $t + k + 1$. The residual $+h$ is the key structural choice: at initialization ($W_1 = 0$), the head reduces exactly to the LM head, giving a warm start rather than random noise.

Initialization Trick

Setting $W_1 = 0$ and $W_2 =$ LM head copy at initialization ensures every head starts as an exact clone of the LM head. Verification: $\text{SiLU}(W_1 h) = \text{SiLU}(0) = 0$, so the bracket reduces to h, and $W_2 h =$ LM head(h). Confirmed at runtime with `torch.allclose` to tolerance 10^{-3} (float16 precision limit).

Training

The backbone is frozen; only the $K = 4$ heads train. For head k with shift $s = k + 1$, labels are input_ids$[:, s:]$ and logits are head_logits$[k][:, :-s, :]$. The total loss is:

$$\mathcal{L} = \sum_{k=0}^{K-1} 0.8^k \cdot \mathcal{L}_k$$

The 0.8^k weighting down-weights far heads because (1) their loss is noisier and (2) acceptance is sequential – a wrong head-0 prediction ends the round regardless of how accurate heads 1–3 are. Trained on UltraChat 25k examples, 2 epochs, backbone on `cuda:0` in float16, heads on `cuda:1` in float32 (float16 heads produce NaN due to softmax overflow at vocab size 128,256). 8-bit Adam (bitsandbytes) used for optimizer memory efficiency.

Greedy Decoding Results

Table 2: Three-way benchmark. Hardware: 2x RTX A5000. 100 tokens, 3 runs.

Config	Notes	tok/s	Speedup
Naive 8B baseline		37.9	1.00x
SpecDecode K=4	1B instruct draft	44.4	1.17x
Medusa greedy (float32 heads)	epoch1 checkpoint	24.6	0.65x
Medusa greedy (float16 heads)	epoch1 checkpoint	26.4	0.69x
Medusa tree attention (width=2)	pending	–	–

Why Greedy Medusa is Slow: The 3-Pass Problem

Medusa greedy with 2.22 tokens/round acceptance should be faster than naive, but it runs at 0.69x. The reason is structural: our greedy implementation does 3 backbone passes per round:

1. **PROPOSE:** Feed last token, get hidden state h for the heads, get backbone prediction at position 0.
2. **VERIFY:** Feed $K-1$ candidates to get backbone predictions at positions 1 through $K-1$.
3. **CACHE UPDATE:** Re-feed accepted tokens into a clean cache snapshot to advance state correctly.

The cache update is forced by a `DynamicCache` mutation bug in transformers 4.56.2: the KV cache is modified in place during the verify pass, contaminating it with rejected tokens. The solution – snapshotting the cache before verify – is correct but adds the third pass.

Expected speedup from the cost model: $2.22/3 \approx 0.74$x. Measured 0.69x, with the gap explained by cross-GPU PCIe latency (h crosses `cuda:0` $\rightarrow$ `cuda:1` every round).

Tree Attention

Motivation

The 3-pass bottleneck is not caused by poor acceptance – 2.22 tokens/round is reasonable. It is caused by verification structure. Even with perfect acceptance (K=4 every round), 3 passes gives at most $4/3 \approx 1.33$x. Tree attention collapses all 3 passes into approximately 1.

Construction

Instead of one candidate chain $[c_0, c_1, c_2, c_3]$, keep the top-2 from each head. This yields a binary tree of depth 4: $2 + 4 + 8 + 16 = 30$ nodes, $2^4 = 16$ candidate paths.

All 30 nodes are flattened into a single sequence and fed to the backbone in one pass with a custom attention mask. The mask enforces: node i can attend to node j if and only if

j is an ancestor of i in the tree (or $j = i$). Siblings – nodes on different branches – are blocked from attending to each other because they represent mutually exclusive futures. Allowing cross-branch attention would contaminate the backbone's logit for node c (child of a) with information from branch b, producing a logit that does not correspond to any real path.

The full attention mask has shape $[1, N, L+N]$ where L is the context length and $N = 30$. The left L columns are all ones (every candidate can see the full context). The right $N \times N$ block is the tree mask built from parent indices.

Expected Gain

With 1 pass instead of 3 and the same 2.22 tokens/round acceptance:

$$\text{speedup} \approx \frac{2.22}{1} \approx 2.2\text{x}$$

In practice, the tree pass processes 30 tokens (more compute than 1), so the real expected speedup is 1.5–2.0x. Benchmark pending.

System

The complete system includes a FastAPI server exposing two endpoints: `POST /generate` for synchronous generation and `WS /stream` for token-by-token streaming. A single-file vanilla JavaScript frontend visualizes tokens live, colored by acceptance status (blue for accepted draft tokens, dark red for backbone corrections). The WebSocket sends each token as {`text, accepted`} JSON so the frontend can color without any additional state.

What I Learned

A few things that were not obvious before building this:

Model size does not determine draft cost. The 1B draft costs 40% of the 8B target per token, not the expected 12.5%. Single-token forward passes are dominated by fixed overhead (kernel launch, memory bandwidth for weights), not by multiply-accumulate FLOPs. This is why the cost model uses $c = 0.40$ instead of $c = 0.125$.

Draft-target dialect matching matters as much as size. Switching from the base 1B to the instruct 1B raised K=4 acceptance from 2.45 to 3.05 tokens/round – a bigger gain than any hyperparameter tuning. Two models with different training objectives operate in different output distributions even at the same positions.

The DynamicCache mutation bug. Transformers' `DynamicCache` is modified in place during every forward pass. Passing the same object to two sequential calls contaminates the second call's computation. The fix – serializing and deserializing through the legacy tuple format to create a fresh object – is not documented anywhere obvious. Found it by debugging why the cache version and the non-cache version diverged after the first round.

Float16 NaN in Medusa training. Training Medusa heads in float16 produces NaN from epoch 0. The softmax over 128,256 tokens computes e^x for logit x; at early training,

logits for some tokens exceed 11 and $e^{11} \approx 60,000$, which overflows float16's maximum of 65,504. The fix is mixed precision: backbone stays float16 (no gradients), heads train in float32.

References

References

[1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. *Fast Inference from Transformers via Speculative Decoding.* ICML 2023. arXiv:2211.17192

[2] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads.* ICML 2024. arXiv:2401.10774

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Re. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. arXiv:2205.14135